

## Билеты

1. Данные. Информация. Алгоритм. Программа. Программное обеспечение.
2. Области применения параллельного программирования.
3. Вычислительный эксперимент.
4. Приближенное извлечение квадратного корня.
5. О понятиях «метод» и «алгоритм».
6. О линейризации в методе Герона и методе Ньютона.
7. О влиянии погрешности округлений в методе Герона.
8. Погрешности вычислений.
9. Архитектура компьютера и принципы фон Неймана.
10. Иерархия памяти компьютера.
11. Этапы параллелизма в архитектуре вычислительных систем.
12. Необходимость и значимость параллельных вычислений.
13. Понятие параллельных вычислений.
14. Параллелизм на одном независимом вычислителе.
15. Пути достижения параллелизма.
16. Систематика Флинна.
17. Мультипроцессоры и мультикомпьютеры.
18. Топология вычислительной сети.
19. Модель параллельных вычислений.
20. Параллельная форма алгоритма.
21. Схема сдваивания.
22. Информационная структура и графовые модели вычислений.
23. Концепция неограниченного параллелизма и схема сдваивания.
24. Построение графов параллельных форм.
25. Параллельный алгоритм. Модель и время выполнения.
26. О временной диаграмме алгоритма.
27. Модифицированная схема сдваивания
28. Оценка эффективности параллельных вычислений.
29. Закон Амдаля.
30. Закон Густавсона-Барсиса
31. Масштабируемость параллельных вычислений.
32. Методика разработки параллельных алгоритмов.
33. Гравитационная задача  $n$  тел.
34. Метрика, метрическое пространство, шар.
35. Сходимость по метрике. Эквивалентные метрики. Открытое и замкнутое множество. Окрестность.
36. Фундаментальная последовательность и ее свойства.
37. Полнота. Изометрия. Пополнение.
38. Оператор. Непрерывность. Условие Липшица.
39. Неподвижная точка. Сжатие. Итерационный метод.

#### 40. Принцип сжимающих отображений

41. Следствия, гарантирующие выполнение принципа сжимающих отображений.

#### 42. О приближенном решении нелинейных уравнений.

#### 43. Метод половинного деления

## 1. Данные. Информация. Алгоритм. Программа. Программное обеспечение.

Сообщение --- сведения и факты, представленные в определённой форме и предназначенные для передачи

Формализация --- описание, являющееся достаточно точным для его однозначной интерпретации

Данные --- представление сообщений в формализованном виде, пригодном для передачи и обработки

Информация --- смысл, придаваемый данным при их представлении

---

Способ --- конечная последовательность однозначно определенных правил, задающих порядок и содержание действий после выполнения конечного числа которых будет получен требуемый результат, если такой существует

Правила элементарны и могут быть выполнены механически исполнителем алгоритма

Механически == для выполнения не требуется ни понимания, ни искусства, ни изобретательности

Алгоритм --- способ решения любой массовой задачи из некоторого класса задач

Алгоритм --- последовательность правил для преобразования исходных данных за конечное число шагов в результат решения любой задачи из рассматриваемого класса задач

---

Программное обеспечение --- логически связанная совокупность компьютерных программ и данных, снабженных документацией

Программный продукт --- программное обеспечение которое может быть продано потребителю

Программная инженерия --- инженерная дисциплина, связанная со всеми аспектами производства ПО, от разработки спецификации требований до поддержки ПО после сдачи в эксплуатацию

## 2. Области применения параллельного программирования.

Области применения:

1. Постановка **вычислительных экспериментов** с целью принятия решений или проверки гипотез
  1. Математическое моделирование позволяет изучать объекты, для которых натурные эксперименты слишком дороги/опасны
2. Методы глубокого обучения языковых моделей
  1. Появление CUDA позволило использовать алгоритмы, известные ещё за 20 лет до этого, для которых не хватало вычислительных мощностей
3. Big Data --- выявление неочевидных закономерностей в больших объемах данных
4. Вычислительные мощности как геополитический ресурс
  1. Страна, которая хочет быть самой крутой, должна быть самой крутой в вычислениях
  2. По аналогии с ядерным оружием, освоением космоса, атомной энергетикой

---

См. также **12. Необходимость и значимость параллельных вычислений.**

### 3. Вычислительный эксперимент.

--- подход к изучению объектов реального мира

---

Три подхода к изучению окружающего мира:

1. Теория
2. Практика
3. Математическое моделирование

Вычислительный эксперимент --- разновидность третьего

---

Цель --- изучение свойств и проверка гипотез без непосредственного взаимодействия с реальным объектом  
Вычислительный эксперимент применяют, когда натурный эксперимент невозможен или слишком опасен/дорог/долго

Сущность (по Самарскому):

- Строится математическая модель объекта --- совокупность математических отношений, отражающая свойства объекта
  - Описывается метод (алгоритм), согласно которому путём взаимодействия с моделью проверяется гипотеза
  - Пишется программа --- реализация метода
- 

#### ТИП

#### AI SLOP

Проведение эксперимента не заканчивается одним прогоном программы. Если результаты расчётов расходятся с ожидаемым поведением объекта (или с данными натурных наблюдений), необходимо **уточнить модель** --- вернуться к первому этапу, пересмотреть исходные предположения, добавить новые факторы или изменить математические соотношения. Таким образом, триада «модель--метод--программа» образует итерационный цикл познания, характерный для всего современного математического моделирования.

### 3+. Триада Самарского

--- подход к математическому моделированию

Есть исследуемый объект и есть ``триада`` для его исследования:

1. Модель
2. Метод
3. Программа

Модель --- совокупность математических отношений, отражающая свойства исследуемого объекта

- Например, система дифференциальных уравнений, которая описывает движение веса на пружине
- Модель формализует интересующие нас свойства

Метод (или алгоритм) --- последовательность вычислительных операций, которые нужно произвести, чтобы найти искомые величины с заданной точностью

- Например, метод вариации постоянной для решения линейного неоднородного дифференциального уравнения первого порядка
- Метод описывает последовательность шагов, которые нужно выполнить, чтобы исследовать свойства объекта в конкретной задаче

Программа --- реализация метода для вычисления на ЭВМ

- Например, код на питоне, который интегрирует определенный вид дифуров
- Программа описывает метод на понятном ЭВМ языке

Постановка эксперимента позволяет уточнять модель в случае, если полученный результат расходится с ожидаемым

#### QUOTE

Сама постановка вопроса о математическом моделировании какого-либо объекта порождает чёткий план действий. Его можно условно разбить на три этапа: Модель--Алгоритм--Программа. На первом этапе выбирается (или строится) эквивалент объекта, отражающий в математической форме важнейшие его свойства -- законы, которым он подчиняется, связи, присущие составляющим его частям, и т. д. Математическая модель (или её фрагменты) исследуется теоретическими методами, что позволяет получить важные предварительные знания об объекте. Второй этап -- выбор (или разработка) алгоритма для реализации модели на компьютере. Модель представляется в форме, удобной для применения численных методов. Определяется последовательность вычислительных и логических операций, которые нужно произвести, чтобы найти искомые величины с заданной точностью. Нельзя, чтобы вычислительные алгоритмы искажали основные свойства модели и, следовательно, исходного объекта, они должны быть экономичными и адаптирующимися к особенностям решаемых задач и используемых компьютеров. На третьем этапе создаются программы, переводящие модель и алгоритм на доступный компьютеру язык. К ним также предъявляются требования экономичности и адаптивности. Их можно назвать электронным эквивалентом изучаемого объекта, уже пригодным для непосредственного испытания на экспериментальной установке -- компьютере.

## 4. Приближенное извлечение квадратного корня.

**Метод Герона** --- численный метод приближенного вычисления квадратного корня

Формула:

$$x_{n+1} := \frac{1}{2} \left( x_n + \frac{S}{x_n} \right)$$

где  $S$  --- неотрицательное число, корень которого хотим приблизить

В качестве  $x_0$  часто берут целое, квадрат которого близок к  $S$

**Метод Ньютона** Метод Ньютона ищет корень нелинейного уравнения  $f(x) = 0$  Метод Герона рассматривают как частный случай Метода Ньютона при  $f(x) = x^2 - S$

**Метод половинного деления** Аналогично, можно записать уравнение  $x^2 - S = 0$  и с помощью метода половинного деления найти  $x$

Метод половинного деления гарантированно сходится к корню, но делает это с линейной скоростью (относительно знаков после запятой) Метод Ньютона (и, соответственно, метод Герона) имеют квадратичную сходимость (относительно знаков после запятой)

## 5. О понятиях «метод» и «алгоритм».

Метод --- набор правил, задающий порядок действий, последовательное выполнение которых позволяет получить требуемый результат, причём

- Набор правил конечен
- Набор правил однозначно определен (не допускает двоякого трактования)

- Правила могут быть выполнены механически (не требуется ни изобретательности, ни понимания, ни искусства)
- Результат гарантировано получается за конечное время (алгоритм не может зависнуть, разве что сообщить об ошибке)

---

Алгоритм --- метод решения любой массовой задачи из некоторого класса задач

## 5+. Численные методы и аналитические методы

--- два подхода к решению математических задач, которые часто противопоставляют друг другу

### Численные методы

#### QUOTE

Численные методы --- методы вычислений, сводящиеся к арифметическим и логическим действиям, которые компьютер выполняет с заданной точностью

Идея:

- Выбирается начальная точка  $x_0$
- Описывается формула (алгоритм), согласно которой можно получить  $x_{k+1}$ , зная  $x_k$
- При этом последовательность  $\{x_k\}_{k=0}^{\infty}$  сходится к истинному значению
  - Иногда для этого требуются дополнительные условия
- Формула применяется, пока не найдено значение желаемой точности

$x_0, x_1, \dots$  называют приближениями

---

Примеры численных методов:

- Метод Герона
- Метод Ньютона

---

Три особенности численных методов:

1. Линеаризация (сведение нелинейной задачи к последовательности линейных)
2. Появление нового параметра, которого не было в исходной задаче --- номера итерации
3. Требование устойчивости --- ошибки округления, накапливающиеся по мере итераций, не должны испортить результат

**Аналитические методы** Идея:

- Выводится общая формула (алгоритм) решения задачи
- В формулу подставляются известные величины
- После упрощения сразу получается правильный ответ (истинное значение)

Примеры аналитических методов:

- Нахождение комплексных корней квадратного трёхчлена через дискриминант
- Дифференцирование по таблице производных
- Нахождение расстояния от точки до плоскости по одноимённой формуле

**Основная проблема**

- Многие задачи не разрешимы аналитически
- Некоторые задачи, разрешимые аналитически, слишком медленно считаются по общей формуле

## 6. О линеаризации в методе Герона и методе Ньютона.

Линеаризация --- сведение нелинейной задачи к последовательности линейных задач

---

Метод Ньютона --- численный метод приближенного нахождения корней уравнения вида  $f(x) = 0$

Метод Герона --- частный случай метода Ньютона; численный метод вычисления приближенного значения квадратного корня

---

**Линеаризация с точки зрения матана** Допустим, мы уже получили приближение  $x_n$  и теперь хотим через него выразить приближение  $x_{n+1}$

Рассматриваем разложение функции  $f$  в ряд Тейлора в окрестности точки  $x_n$

$$f(x) = f(x_n) + f'(x_n)(x - x_n) + \frac{1}{2}f''(x_n)(x - x_n)^2 + \dots$$

Отбрасываем нелинейную часть, получаем:

$$f(x) \approx f(x_n) + f'(x_n)(x - x_n)$$

Мы ищем решение уравнения  $f(x) = 0$ , поэтому приравниваем правую часть к нулю:

$$f(x_n) + f'(x_n)(x - x_n) = 0$$

Выражаем  $x$ :

$$x = x_n - \frac{f(x_n)}{f'(x_n)}$$

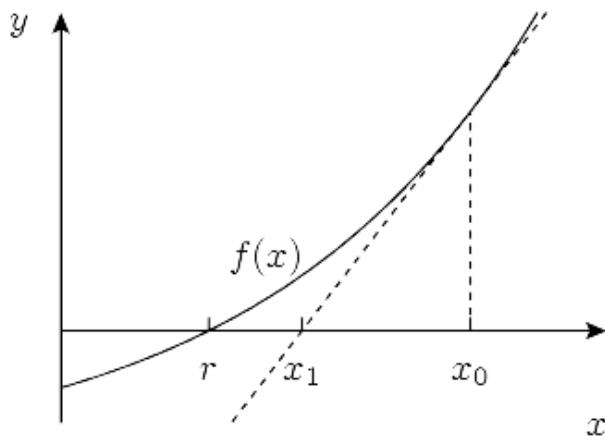
Получили формулу, согласно которой, зная очередное приближение  $x_n$ , можно получить некоторый  $x$ , близкий к искомому решению уравнения  $f(x) = 0$ , то есть следующее приближение  $x_{n+1}$

$$x_{n+1} := x_n - \frac{f(x_n)}{f'(x_n)}$$

---

### Линеаризация с точки зрения геометрии

- Выберем точку на оси абсцисс --- начальное приближение  $x_0$
- Проведём касательную к графику в точке  $(x_0, f(x_0))$
- Пусть  $x_1$  --- точка пересечения касательной с осью абсцисс
- $x_1$  будет ближе к искомому корню, чем  $x_0$
- Поэтому мы возьмём  $x_1$  в качестве следующего приближения и повторим процесс



**Как отсюда вывести формулу?** Допустим, мы уже получили приближение  $x_n$  и теперь хотим через него выразить приближение  $x_{n+1}$

Нам нужно найти точку  $x_{n+1}$  пересечения двух прямых:

1. Касательной к функции  $f$  в точке  $x_n$ 
  1. Её уравнение  $y(x_{n+1}) = f(x_n) + f'(x_n)(x_{n+1} - x_n)$
2. Оси абсцисс
  1. Её уравнение  $y \equiv 0$

Приравняв правые части, получаем

$$f(x_n) + f'(x_n)(x_{n+1} - x_n) = 0$$

Выражаем отсюда  $x_{n+1}$

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$


---

#### Замечание

1. Здесь довольно сильные ограничения на  $f$  для сходимости последовательности  $\{x_n\}_{k=1}^{\infty}$  к корню  $f(x) = 0$ 
  - Без выполнения этих ограничений в общем случае метод не будет работать
2. В частности, если на очередном шаге в точке  $x_n$  производная функции  $f$  обращается в ноль, то продолжить построение последовательности не получится
  - Здесь может помочь переВыбор начального приближения

Достаточные условия сходимости (условия Фурье):

1. Корень  $f(x) = 0$  существует (на области поиска)
2. Функция  $f$  строго монотонна (на всей области поиска)
3. У функции  $f$  нет точек перегиба (на всей области поиска)
4.  $f(x_0) \cdot f''(x_0) > 0$ 
  1. Нужно правильно выбрать начальное приближение: так, чтобы знак  $f$  совпадал со знаком её второй производной

Бывают и другие условия сходимости --- для более частных и более общих случаев

## 7. О влиянии погрешности округлений в методе Герона.

Короткий ответ:

- Погрешности округления не накапливаются лавинообразно и не ломают метод

- Это обеспечивается тем, что метод Герона обладает мощным свойством самокоррекции

---

Формула Герона вычисления приближенного значения  $\sqrt{a}$ :

$$x_{n+1} = \frac{1}{2} \left( x_n + \frac{a}{x_n} \right)$$

---

Упрощённое рассуждение:

- На каждом шаге мы берём среднее арифметическое двух значений:
  1.  $x_n$
  2.  $\frac{a}{x_n}$ , которое обратно-пропорционально к  $x_n$
- Если  $x_n$  чуть больше точного корня, то  $\frac{a}{x_n}$  чуть меньше, и наоборот
- При усреднении эти отклонения взаимно компенсируют друг друга: приближение не искажается сильнее при очередном округлении
- Поэтому их среднее арифметическое всегда лежит ближе к корню, чем текущее приближение

---

Численные методы, которые, подобно методу Герона, не подвержены искажению результата из-за многочисленных округлений, называют устойчивыми методами

## 8. Погрешности вычислений.

Погрешность --- отклонение значения, полученного в результате вычислений, от истинного значения

---

Пусть  $x$  --- истинное значение,  $x^*$  --- вычисленное значение

Абсолютная погрешность --- модуль разности вычисленного значения и истинного значения

$$\Delta x = |x^* - x|$$

Относительная погрешность --- абсолютная погрешность, деленная на модуль истинного значения:

$$\delta x = \frac{\Delta x}{|x|} = \frac{|x^* - x|}{|x|}$$

Приведенная погрешность --- абсолютная погрешность, деленная на нормирующий множитель

$$\gamma x = \frac{\Delta x}{\mu} = \frac{|x^* - x|}{\mu}$$

где  $\mu$  может быть например  $\mu = |x_{\max} - x_{\min}|$  или  $\mu = |x_{\max}|$  или что-то ещё

---

Вопрос: как узнать погрешность, если истинное значение неизвестно? Ответ: погрешность можно оценить

Априорный --- полученный до (постановки эксперимента) Апостериорный --- полученный после или во время (постановки эксперимента)

В случае метода последовательных приближений:

- Априорная оценка --- ответ на вопрос ``сколько итераций нужно сделать, чтобы получить заданную точность" (до того как итерации начаты)
- Апостериорная оценка --- ответ на вопрос ``можно ли уже остановиться?" (на текущий момент сколько-то итераций сделано)

Для каждой задачи априорная и апостериорная оценка вычисляется по собственной формуле Априорная получается более грубой (слишком пессимистичная) Апостериорная --- более точной



## 9. Архитектура компьютера и принципы фон Неймана.

Архитектура --- совокупность наиболее важных решений об организации системы

Два способа организации архитектуры:

1. Структурная декомпозиция
    1. Выделение отдельных функциональных блоков, из которых система состоит (ALU, регистры, декодер, ...)
  2. Иерархическая декомпозиция
    1. Выделение уровней абстракции (BLP, OLP, ILP, ...)
- 

Принципы фон Неймана:

1. Принцип однородности и адресуемости памяти
  1. Однородность:
    1. Данные и команды хранятся в одной памяти
    2. Признаки, позволяющие их явно различать, отсутствуют
2. Принцип последовательного программного управления
3. Принцип запоминаемой программы
4. Принцип параллелизма на аппаратном уровне
  1. Одновременная работа со всеми разрядами одного машинного слова
5. Принцип использования двоичной системы счисления
6. Принцип иерархии запоминающих устройств

## 10. Иерархия памяти компьютера.

В случае CPU:

1. Регистры
  1. В отличие от других типов памяти, обычно специализированы:
    1. Регистры общего назначения
      1. Для адресной, целочисленной и логической арифметики
    2. Регистры для работы с числами с плавающей точкой
    3. Векторные регистры
    4. Регистры с флагами
    5. RSP, RBP
  2. Типичный размер:
    1. от 128 бит до 512 бит для векторных регистров
    2. 32 бита или 64 бита для остальных
  3. Обычно реализуются с помощью триггеров
2. Кеши
  1. Обычно их три уровня:
    1. L1 --- локальный кеш ядра
      1. Типичный размер: 64 KiB
      2. Иногда специализирован: разделен на кеш для инструкций и кеш для данных
    2. L2 --- обычно тоже локальный для ядра
      1. Типичный размер: 1 MiB
    3. L3 --- общий кеш для всех ядер на одном кристалле

1. Типичный размер: 32 MiB
2. Обычно реализуются с помощью SRAM
3. Оперативная память
  1. Типичный объем: единицы-десятки GiB
  2. Обычно реализуется с помощью DRAM
  3. Основные характеристики --- тактовая частота и тайминги
4. Вторичная память
  1. Типичный объем: от сотен GiB до десятков TiB
  2. Наиболее распространенные технологии:
    1. Технологии хранения: Flash NAND, HDD
    2. Интерфейсы подключения: NVME, SATA
5. "Третичная память"
  1. Иногда выделяют как отдельный уровень
  2. Сюда относятся:
    1. Холодные хранилища: библиотеки магнитных лент
    2. Сетевые файловые системы

## 10+. Иерархия памяти в случае GPU

Большинство современных систем общего назначения гетерогенные У графических ускорителей своя иерархия памяти, сильно отличающаяся от иерархии в случае CPU

---

GPU --- мультипроцессорные системы

- В современных GPU от Nvidia количество процессоров измеряется десятками-сотнями
  - Один такой процессор называют SM (Streaming Multiprocessor)
  - В RTX4090: 128 SM
- Внутри одного процессора могут работать до нескольких тысяч потоков, объединяемых в блоки (Thread Block)

---

Физические уровни памяти:

1. Регистры
  1. Локальная память одного SM
    1. У каждого SM свой регистровый файл
  2. Физических регистров очень много
    1. Пример для RTX4090:
      1. 8,3 млн всего
      2. 65536 на один мультипроцессор (SM)
2. Shared Memory
  1. Внутри одного SM помимо регистрового файла есть shared SRAM массив
  2. Хотя физически это один модуль, логически он обычно делится на два:
    1. L1 cache --- локальный кеш SM (управляется аппаратно)
    2. Программируемая разделяемая память
      1. Для каждого Thread Block в этом массиве будет свой кусок разделяемой памяти (если ему он нужен)
    3. К тому же, это разделение конфигурируется программно

### 3. L2 cache

1. Управляется аппаратно
2. Реализуется с помощью SRAM
3. Общий кеш для всех SM
4. Размер измеряется единицами-десятками MiB, гораздо реже --- сотнями MiB
  1. В RTX4090: 72 MiB

### 4. VRAM

1. Управляется программно
2. Реализуется с помощью DRAM
3. Общая память для всех SM
4. Объем измеряется единицами-десятками GiB, гораздо реже --- сотнями GiB
  1. В RTX4090: 24 GiB

### 5. Разделяемая с CPU память

1. Иногда GPU и CPU делят одни и те же физические DRAM модули
2. Примеры: Apple Silicon, Nvidia DGX Spark

## 11. Этапы параллелизма в архитектуре вычислительных систем.

Уровни параллелизма в аппаратуре:

- на уровне битов (BLP)
  - вычислитель выполняет операцию над всеми битами одного машинного слова одновременно
  - ``одновременно`` == за один такт обрабатывает все разряды
- на уровне операндов (OLP)
  - векторные инструкции
    - \* одна инструкция вычислителя позволяет применить преобразование над целым набором операндов
    - \* частный случай SIMD
- на уровне команд (ILP)
  - суперскалярность и конвейеризация
    - \* одновременное выполнение нескольких инструкций одной программы
    - \* вычислитель должен иметь дополнительные функциональные устройства
  - Out-Of-Order Execution
    - \* выполнение инструкций не в программном порядке, а в порядке готовности операндов с отложенной записью результата
  - VLIW, EPIC
    - \* Архитектуры с явным параллелизмом
- параллелизм на уровне задач (TLP) и данных (DLP)
  - параллелизм с общей памятью
    - \* многоядерные CPU, многопроцессорные ЭВМ, GPU
  - **параллелизм с распределенной памятью**
    - \* кластеры, grid-технологии, облачные и туманные вычисления

## 12. Необходимость и значимость параллельных вычислений.

Необходимость применения параллелизма --- физические ограничения

- Ограничение по частоте одного вычислительного ядра

- Начиная с 4-5 ГГц дальнейшее увеличение частоты приводит к непропорциональному росту затрат энергии на охлаждение
- Это не позволяет увеличивать производительность просто повышая тактовую частоту одного ядра
- Ограниченный рост производительности по мере увеличения числа транзисторов в одном ядре
  - До какого-то момента уменьшение размера транзисторов приводило к уменьшению тепловыделения и энергопотребления --- можно было наращивать мощность одного ядра за счёт этого
  - Сейчас наоборот --- дальнейшее уменьшение размера транзисторов приводит к непропорциональному росту тепловыделения

Параллелизм приходится использовать в силу того, что не получается бесконечно наращивать производительность одного вычислительного ядра

---

Значимость применения параллелизма:

- Быстрее решаются те же задачи
  - Более энергоэффективно решаются те же задачи
- 

См. также [2. Области применения параллельного программирования.](#)

## 13. Понятие параллельных вычислений.

Параллельные вычисления --- способ организации вычислений, при котором создаются несколько потоков управления

Конкурентные вычисления --- способ организации вычислений, при котором несколько задач одновременно активны на одном вычислителе

Распределенные вычисления --- способ организации вычислений, при котором вычисления происходят на множестве вычислителей, соединенных сетью передачи данных

## 13+. Классификация распределенных вычислений

Классификация распределенных вычислений:

- Кластерные технологии
  - Кластер --- объединение вычислителей для решения общих задач, при этом
    - \* Вычислители объединены локальной сетью (нет географической распределенности)
    - \* Система принадлежит одной организации (централизована)
    - \* Вычислители зачастую сильно похожи
- Grid технологии
  - Grid --- объединение вычислителей для решения общих задач, при этом
    - \* Вычислители объединены глобальной сетью и географически удалены друг от друга
    - \* Система может принадлежать независимым организациям (децентрализована)
      - В случае если они делятся своими ресурсами или совместно решают общую задачу
    - \* Вычислители могут сильно отличаться
- Облачные технологии
  - Модель предоставления вычислительных ресурсов ``по требованию"
  - Предоставление доступа к инфраструктуре осуществляется через веб-сервис
  - Все вычисления происходят на стороне облака
  - Централизованная (принадлежит одной организации), но географически распределенная инфраструктура (обычно в виде набора кластеров)

- Туманные вычисления
  - Дополнение облачных вычислений:
    - \* Вычислительные узлы располагаются географически близко к конечному пользователю
    - \* На туманных узлах происходят вычисления и делается ответ пользователю, а в облако отправляется только результат вычислений
    - \* Это позволяет уменьшить задержки для конечного пользователя
  - Организация:
    - \* Конечные устройства (edge)
      - Например камеры видеонаблюдения
    - \* Туман (fog)
      - Локальные (по отношению к edge) вычислители
    - \* Облако (cloud)
      - Удаленные вычислители

## 14. Параллелизм на одном независимом вычислителе

Механизмы параллелизма на одном вычислительном ядре:

- Одновременная (за один такт) обработка сразу всех битов одного машинного слова
- Векторные операции
  - Применение одной и той же инструкции сразу к нескольким операндам
  - Несколько операндов размещаются в одном широком (128-512 бит) регистре, операция применяется сразу ко всем
  - Примеры: Intel AVX, ARM NEON
- Суперскалярность
  - Множество функциональных блоков одного типа, которые могут быть задействованы одновременно
  - Может быть несколько полноценных конвейеров, каждый со своим планировщиком
- Конвейеризация
  - Выполнение разных этапов разных инструкций одновременно
- Аппаратная многопоточность
  - Создание нескольких потоков управления на одном вычислительном ядре и переключение между ними
  - Примеры: Intel Hyperthreading, AMD Multithreading

---

Многопроцессорность и многоядерность --- примеры параллелизма на уровне одной ЭВМ

## 15. Пути достижения параллелизма.

Пути достижения параллелизма на уровне аппаратуры:

- Векторизация
  - Параллелизм уровня операндов (OLP)
  - Одна и та же операция применяется сразу к массиву данных
- Конвейеризация
  - Параллелизм уровня инструкций (ILP)
  - Этапы нескольких разных операций выполняются одновременно на специализированных вычислителях
- Суперскалярность

- Параллелизм уровня инструкций (ILP)
- Несколько аппаратных модулей одного типа
  - \* В том числе несколько полноценных конвейеров
- Спекулятивное исполнение
  - Параллелизм уровня инструкций (ILP)
  - Процессор пытается угадать ветку и начинает её прогонять через конвейер до того, как точно известно, какая ветка на самом деле нужна
  - Если не угадал, делает сброс конвейера
- Специализированные архитектуры с явным параллелизмом уровня инструкций
  - Примеры: VLIW, EPIC
  - Инструкции собираются в группы, каждая группа выполняется параллельно
  - Сложнее писать компиляторы под это, но зато железо становится попроще

---

Пути достижения параллелизма на уровне программ:

- Многопоточность с общей памятью
  - Создание нескольких потоков управления и их работа в общем адресном пространстве
  - Обмен данными с помощью разделяемых переменных
  - Модель 1:1:
    - \* Одному пользовательскому потоку соответствует один поток ОС
    - \* Примеры:
      - OpenMP
      - `std::thread` в C++
      - Классические потоки Java
  - Модель M:N:
    - \* Мультиплексирование корутин на потоки ОС в managed runtime средах
    - \* Такое есть в Go, Kotlin, C# и ещё много где
- Распределенные вычисления
  - Создание нескольких процессов, каждый со своим адресным пространством
  - Обмен данными с помощью сообщений
  - Пример: MPI
- GPGPU
  - Перенос общих вычислений на специализированные ускорители
  - ``Возведение идеи параллелизма по данным в абсолют``
  - Сюда относятся:
    - \* CUDA
    - \* OpenCL

## 16. Систематика Флинна.

--- подход к классификации архитектур вычислительных систем

---

### 1. SISD

1. Один поток управления и один поток данных
2. Работа на одном ядре без векторных расширений

## 2. SIMD

1. Один поток управления и множество потоков данных
2. Одна операция применяется сразу к массиву данных
3. Основной пример --- векторные инструкции CPU/GPU
  1. В широком регистре размещается сразу несколько значений
  2. За один такт применяется операция сразу ко всем значениям в регистре
4. Технологии:
  1. x86\_64: AVX
  2. ARM64: NEON
  3. RISC-V: расширение V

## 3. MISD

1. Множество потоков управления и один поток данных
2. Практически не встречается на практике
3. Добавляется к классификации скорее ради полноты изложения
4. Пример --- системы повышенной надежности
  1. Несколько вычислителей, каждый выполняет свою версию программы над одними и теми же данными
  2. Результаты должны получиться одни и те же, но из-за ошибок в программах результаты могут разойтись
  3. Окончательный результат выбирается по принципу большинства

## 4. MIMD

1. Множество потоков управления и множество потоков данных
2. Типичный workflow современной ЭВМ: многоядерность/многопроцессорность + многозадачность

# 17. Мультипроцессоры и мультикомпьютеры.

Мультипроцессоры:

- Вычислители работают с общей памятью (имеют к ней доступ напрямую)
  - Общение посредством изменения значений разделяемых переменных
  - Масштабируемость ограничена пропускной способностью системной шины
  - Необходимость использования примитивов синхронизации и протоколов когерентности кешей
  - Примеры: многоядерные CPU, серверные платы с несколькими сокетами
- 

Мультипроцессоры делят на UMA и NUMA

- UMA (Uniform Memory Access) --- доступ к каждой ячейке памяти каждого процессора примерно одинаковый
  - NUMA (NonUniform Memory Access) --- у каждого вычислителя есть локальная память, доступ к которой быстрее, чем к памяти другого вычислителя или к глобальной памяти (если такая имеется)
- 

Мультикомпьютеры:

- [Архитектура с распределённой памятью](#)
- Множество вычислительных узлов, каждый работает в своем адресном пространстве
- Взаимодействие путём отправки сообщений по сети
- Масштабируемость выше, чем у мультипроцессоров, но и накладные расходы на коммуникацию выше

- На мультикомпьютерах строятся кластеры, grid-технологии, облачные и туманные вычисления

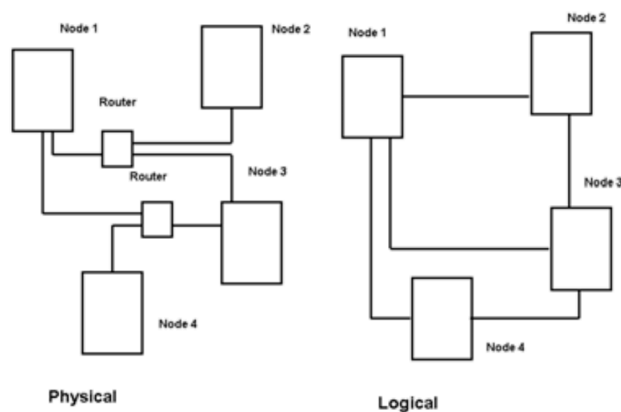
## 18. Топология вычислительной сети.

Топология сети --- структура линий коммуникации между вычислительными узлами

Иногда различают:

- Физическую топологию --- организацию физических линий и сетевых устройств (хабов, свитчей, роутеров)
- Логическую топологию --- организацию потоков данных между узлами

Figure 1 Physical and Logical Topologies



### QUOTE

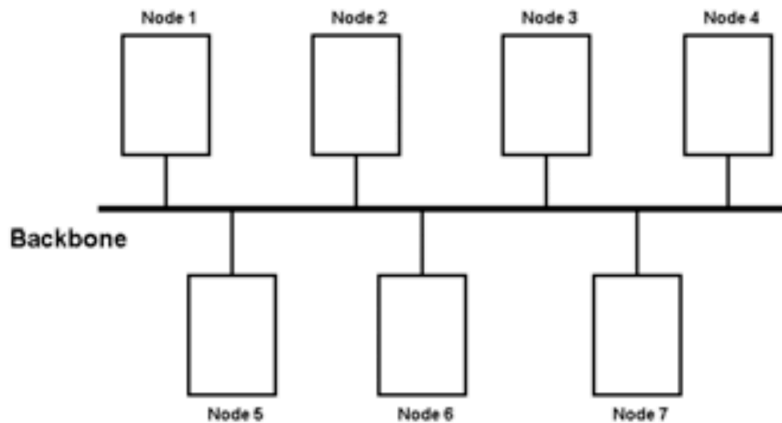
Physical Network Topology emphasizes the hardware associated with the system including workstations, remote terminals, servers, and the associated wiring between assets. Logical Network Topology (also known as Signal Topology) emphasizes the representation of data flow between nodes, not dissimilar from Graph Theory analysis. The logical topography of a network can be dynamically reconfigured when select network equipment, such as routers, is available.

Топология ``шина``

Все узлы сети подключаются к общей шине, обмен данными происходит только с её помощью. Когда узел отправляет сообщение, все другие узлы получают его.



Figure 2 Bus Network Topology



Преимущества:

- Дешёво:
  - Относительно мало физических линий
  - Нет вспомогательных узлов --- только шина
- Отказ отдельного узла никак не влияет на возможность передачи сообщений между остальными узлами
- Если нагрузка на шину небольшая, то практически самый быстрый способ передачи
  - Нет маршрутизации, нет путей как таковых
  - Для бродкаст сообщений ничего быстрее не придумаешь

Недостатки:

- Большая нагрузка на шину
  - Ограничения физического канала не позволят передавать большое количество сообщений одновременно
- Отказ шины приведёт к потере соединения вообще везде
  - Или разделит сеть на несколько частей, между которыми сообщения не получится передать

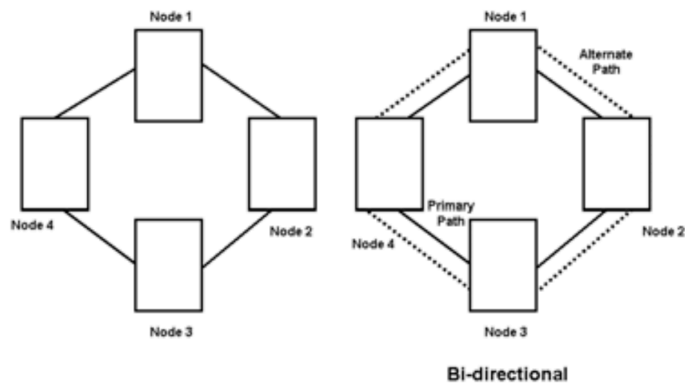
Шину в такой топологии иногда называют позвоночником (backbone)

---

Топология ``кольцо``

Вся сеть --- один простой цикл в графе Каждый узел соединён с двумя другими узлами (при этом нет петель, мультидуг и сеть связна)

Figure 3 Ring Network Topology



Преимущества:

- Относительно дешево:
  - Мало физических линий
  - Нет вспомогательных узлов
- Повреждение одной физической линии не так критично, как в случае с шиной

Недостатки:

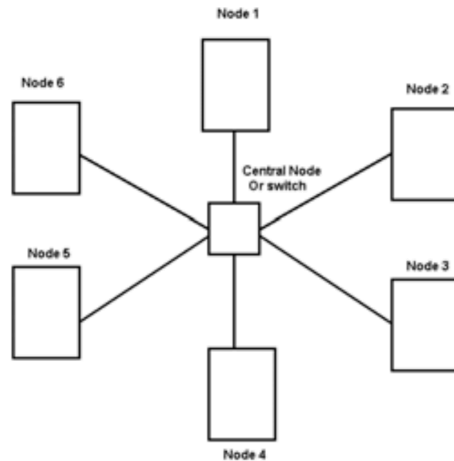
- Большие задержки --- пакеты вынуждены проходить через большое количество промежуточных узлов
- Отказ вычислительного узла так же плох, как повреждение физической линии --- уменьшает доступное количество маршрутов
- В случае ошибки с адресами, пакеты начнут наворачивать круги --- придется поддерживать механизм, который это предотвратит
- Вычислительные узлы вынуждены постоянно пересылать чужие пакеты

---

Топология ``звезда''

Каждый вычислительный узел соединен единственной физической линией со вспомогательным узлом. Вспомогательный узел обеспечивает маршрутизацию.

Figure 4 Star Network Topology



Преимущества:

- Относительно мало физических линий
- Небольшие задержки
  - При условии, что центральный узел справляется с нагрузкой
  - Но так как он вспомогательный --- он специально разработан так, чтобы справляться с большими нагрузками
- Повреждение одной физической линии повлечет за собой потерю соединения со всего одним узлом

Недостатки:

- Есть вспомогательный узел
- Отказ вспомогательного узла влечёт выход из строя всей сети

---

Топология ``полный граф``

Каждый узел соединён с каждым

Преимущества:

- Ничего быстрее не придумаешь
  - (В случае передачи пакета от одного узла к другому)
  - Если это бродкаст сообщение, то самый быстрый вариант --- ``шина``
- Ничего надежнее не придумаешь

Недостатки:

- Безумно дорого и совершенно не масштабируемо

## 19. Модель параллельных вычислений

Проблема: трудно построить универсальную модель параллельного вычислителя

Подход: разделить

- архитектуры с общей памятью
- архитектуры с [распределенной памятью](#)
- архитектуры с иерархической памятью
  - (по всей видимости, имелись в виду гибридные системы: распределенная система, где на каждом узле параллелизм с общей памятью)

Для каждой архитектуры построить отдельную модель, которая будет более-менее универсальной для своего типа параллельных вычислений

---

Одна из наиболее распространенных моделей: модель PRAM (Parallel Random Access Machine) --- модель идеализированного параллельного вычислителя

См. [23. Концепция неограниченного параллелизма и схема сдваивания.](#)

PRAM моделирует синхронную систему с общей памятью

---

#### TIP

AI SLOP

**BSP (Bulk Synchronous Parallel)** --- модель синхронной распределенной/иерархической системы

Вычисления разбиваются на **супершаги**, каждый из которых состоит из трёх фаз:

1. **Локальные вычисления:** каждый процессор выполняет операции над своей локальной памятью.
2. **Коммуникации:** процессоры обмениваются сообщениями (отправка/получение).
3. **Барьерная синхронизация:** все процессоры ждут завершения коммуникаций текущего шага, прежде чем начать следующий.

Три параметра модели:

- Число процессоров
  - Пропускная способность сети
  - Длительность задержек, вызванных синхронизацией
- 

#### TIP

AI SLOP

**LogP** --- модель асинхронной распределенной/иерархической системы

Четыре параметра:

- **L** (latency) --- задержка передачи сообщения
- **o** (overhead) --- время процессора на отправку/приём одного сообщения
- **g** (gap) --- минимальный интервал между двумя сообщениями (обратная пропускная способность)
- **P** --- число процессоров

Нет глобальной синхронизации, модель асинхронная. Хорошо отражает поведение реальных кластеров, но анализ алгоритмов сложнее, чем в BSP.

---

## 20. Параллельная форма алгоритма.

--- способ представления информационных зависимостей задачи в виде графа, называемого графом параллельных форм

---

Контекст:

- На множестве всех операций, которые мы хотим выполнить, задан частичный порядок --- зависимости по данным между операциями
- Циклических зависимостей нет

---

Строим граф:

- Вершины --- операции
- Ребра --- информационные зависимости между операциями
- Иногда добавляют входные данные в качестве листьев

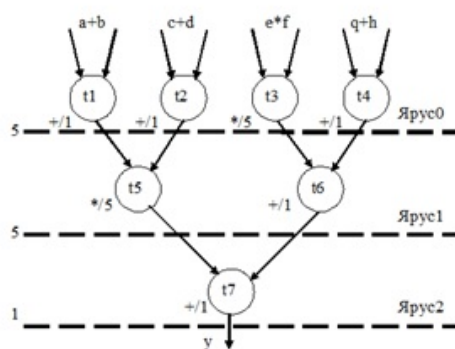
Вершины объединяются в группы, называемые ярусами, причём:

- Между вершинами одного яруса нет рёбер (задачи одного яруса независимы)
- Ярусы линейно-упорядочены зависимостями содержащихся в них вершин (задачи следующего яруса зависят от задач предыдущих ярусов)

Таким образом, каждый ярус --- набор операций, которые можно выполнять параллельно

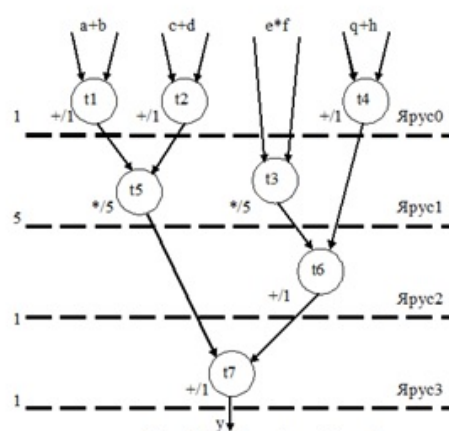
$$y = (((a+b) \times (c + d)) + ((e \times f) + (q + h))).$$

Вариант 1.



$$T = 5 + 5 + 1 = 11$$

Вариант 2.



$$T = 1 + 5 + 1 + 1 = 8$$

Такой граф строится для

- выявления неочевидного параллелизма
- оценки объема коммуникаций

---

Параллельная форма алгоритма не единственна

Естественная задача --- построить параллельную форму алгоритма такую, что

- Высота (количество ярусов) минимальна
- Ширина (количество операций на одном ярусе) ограничена

Такая задача является NP-трудной (любая задача класса NP к ней сводится за полиномиальное время)

---

Также см. [24. Построение графов параллельных форм.](#)

## 21. Схема сдваивания.

Возможно, здесь рассказан частный случай, а не полная картина

---

Схема сдваивания (она же каскадная схема или схема редукции) --- шаблон проектирования параллельных алгоритмов

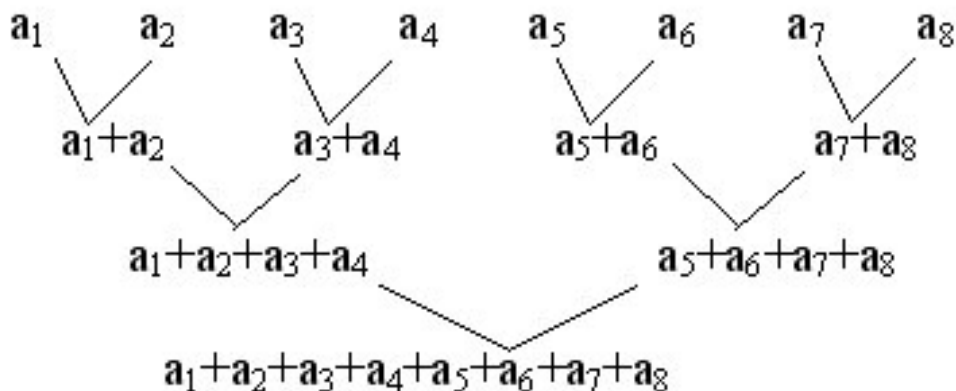
---

Постановка задачи:

- Дана конечная последовательность  $\{a_k\}_{k=1}^m$
- Дана ассоциативная операция  $\oplus$
- Необходимо вычислить  $a_1 \oplus \dots \oplus a_m$

Алгоритм:

- На каждом шаге разбиваем входные данные на пары
- К элементам пары применяем операцию
- Результаты применения операции к каждой паре берём как входные данные следующего шага



## 22. Информационная структура и графовые модели вычислений.

Информационная структура --- структура коммуникации между вычислителями в процессе параллельной работы

Для её представления используется несколько графовых моделей:

1. [20. Параллельная форма алгоритма.](#)
2. Граф ``подзадачи-сообщения``
3. Граф ``процессы-каналы``

**Граф ``процессы-каналы``** Структура:

- Вершины --- процессы
- Ребра --- каналы (очереди сообщений) для передачи данных между процессами

Сущность:

- Показывает физическое отображение алгоритма на конкретную вычислительную систему
- Зависит от числа вычислителей и топологии сети
- Позволяет оценить качество распределения задач по вычислителям

### ТИП

#### AI SLOP

- Сначала строят **граф «подзадачи-сообщения»** -- он отражает максимально детализированный параллелизм алгоритма
- Затем выполняют **агломерацию** (объединение мелких подзадач в более крупные) и **распределение (mapping)** этих укрупнённых подзадач по вычислителям

- В результате получают **граф «процессы--каналы»**:

Таким образом, второй граф -- это проекция первого на конкретное число вычислителей и топологию сети, с учётом объединения подзадач.

#### TIP

AI SLOP

**Граф «подзадачи--сообщения»** -- результат этапов **Partitioning** и **Communication**.

- *Partitioning*: разбиение исходной задачи на мелкие подзадачи.
- *Communication*: выявление информационных зависимостей между ними.

**Граф «процессы--каналы»** -- результат этапов **Agglomeration** и **Mapping**.

- *Agglomeration*: объединение исходных подзадач в более крупные процессы.
- *Mapping*: назначение этих процессов на конкретные вычислители.

## 23. Концепция неограниченного параллелизма и схема сдваивания.

#### TIP

21. Схема сдваивания.

### Концепция неограниченного параллелизма: модель PRAM

#### QUOTE

PRAM --- система, работа которой происходит в течение некоторых единиц дискретного времени, называемых тактами и обладает семью свойствами:

1. система имеет любое нужное число идентичных вычислителей
2. система имеет произвольно большую память и одновременный доступ ко всем вычислителям
3. каждый вычислитель может выполнить одну операцию за такт из множества основных операций
4. время выполнения вспомогательных операций пренебрегается
5. никакие конфликты при обращении к памяти не возникают
6. все входные данные перед началом работы системы находятся в памяти
7. после выполнения работы результаты хранятся в памяти

PRAM (Parallel Random Access Machine) --- модель идеализированной параллельной ЭВМ

- неограниченное количество одинаковых процессоров
  - разделяемая память неограниченного объема
  - процессоры синхронно за такт выполняют одну операцию чтения/записи/вычислений
  - модель полностью игнорирует:
    - задержки при коммуникации
    - конфликты системной шины
- \* считаем, что у неё неограниченная пропускная способность

#### TIP

AI SLOP

С гонками данных (точнее, одновременным доступом к одной ячейке) сложнее:

- Могут ли два вычислителя одновременно обратиться к одной ячейке с целью чтения/записи, зависит от определения вычислителя
- EREW (Exclusive Read, Exclusive Write) --- не могут
- CREW (Concurrent Read, Exclusive Write) --- могут, если среди операций нет записей
- CRCW (Concurrent Read, Concurrent Write) --- могут + несколько подвидов определения, что произойдёт в таком случае
  - *Common* --- все пишут одно и то же значение (иначе операция недопустима)
  - *Arbitrary* --- записывается значение произвольного процессора
  - *Priority* --- записывается значение процессора с наименьшим номером
  - *Combining* --- результат вычисляется из всех записываемых значений (например, сумма)

## 24. Построение графов параллельных форм.

См. [20. Параллельная форма алгоритма](#).

---

Граф параллельной формы не единственен. Поэтому имеет смысл ставить задачу нахождения оптимального/субоптимального графа параллельных форм

---

Граф параллельных форм минимальной высоты без ограничений по ширине строится ``в лоб``:

- В качестве вершин берём операции, в качестве рёбер --- информационные зависимости
  - На очередном шаге рассматриваем все вершины, в которые не входит ни одно ребро
  - Все такие вершины берём в качестве нового яруса
  - Мысленно выбрасываем все вершины нового яруса из графа и повторяем процесс
- 

Построение минимального по высоте графа параллельных форм с ограничением по ширине яруса --- NP-трудная задача

- Любая NP задача к ней сводится за полиномиальное время
  - Универсального полиномиального алгоритма для ДМТ/НМТ для решения этой задачи не существует или он неизвестен
- 

Один из шаблонов, применяемых в субоптимальных алгоритмах построения графа параллельных форм --- List Scheduling:

- Согласно некоторой эвристике, каждой операции назначается приоритет
- Для операций поддерживается очередь с приоритетами
- Для заполнения яруса графа из очереди достаётся операция, у которой:
  1. Все зависимости уже помещены на предыдущие ярусы
  2. Наибольший приоритет среди таких операций

### TIP

AI SLOP

**Распространённые эвристики приоритетов:**

- **HLF (Highest Level First)** --- длина максимального пути от вершины до конца графа
- **CP (Critical Path)** --- длина критического пути через вершину



- **ISF (Most Immediate Successors First)** --- число непосредственных преемников
- **ETF (Earliest Time First)** --- выбирается операция, способная стартовать раньше всех с учётом текущей загрузки процессоров.
- **DLS (Dynamic List Scheduling)** --- приоритеты пересчитываются динамически после каждого назначения.

## 25. Параллельный алгоритм. Модель и время выполнения.

**Модель** Пара  $\langle G, H_p \rangle$  может рассматриваться как модель параллельного алгоритма

- $G$  --- граф [параллельной формы алгоритма](#)
  - Вершины --- операции
  - Ребра --- информационные зависимости между операциями
- $H_p$  --- расписание для  $p$  вычислителей

---

Расписание --- множество троек вида

(номер вычислителя, номер операции, время начала выполнения)

При этом:

- Один и тот же вычислитель не может назначаться двум операциям в один момент времени
- К назначаемому времени выполнения операции все необходимые данные уже должны быть вычислены

Расписание зависит от параллельной формы --- допустимое расписание должно учитывать зависимости  
Параллельная форма от расписания не зависит

---

**Время выполнения** Будем считать, что

- каждая операция выполняется за один такт
- начальное время равно нулю

Тогда можно вычислить следующие величины:

1. Время работы с расписанием  $H_p$ :

$$T_p(G, H_p) = \max_{v \in G} (\text{start\_time}(v) + 1)$$

2. Лучшее возможное время работы для параллельной формы  $G$ :

$$T_p(G) = \min_{H_p} T_p(G, H_p)$$

3. Лучшее возможное время работы на  $p$  вычислителях:

$$T_p = \min_G T_p(G)$$

## 26. О временной диаграмме алгоритма

### QUOTE

Что-то вроде диаграммы Ганта, только задачи для вычислителей, а не для людей

---

См. [25. Параллельный алгоритм. Модель и время выполнения.](#)

Временную диаграмму можно построить статически, тогда это будет визуализацией расписания:

- ось  $x$  --- время (такты)
  - ось  $y$  --- вычислители (или потоки управления)
  - в точке  $(t, i)$  --- идентификатор задачи, выполняемой вычислителем  $i$  в момент  $t$ 
    - или пусто, если вычислитель простаивает
- 

Временную диаграмму строят, чтобы:

- оценить загруженность вычислителей и увидеть простои
  - сравнить разные расписания для одной параллельной формы
  - увидеть последовательные участки
    - в том числе вызванные неудачным расписанием
  - увидеть критический путь
    - (его длина от расписания не зависит, это характеристика графа параллельной формы)
- 

Временную диаграмму также можно построить динамически, собрав профиль исполнения

## 27. Модифицированная схема сдваивания

Возможно, здесь рассказан частный случай, а не полная картина

---

Модифицированная схема сдваивания --- шаблон проектирования параллельных алгоритмов

---

Постановка задачи:

- Дана конечная последовательность  $\{a_k\}_{k=1}^n$
  - Дана ассоциативная операция  $\oplus$
  - Необходимо вычислить  $a_1 \oplus \dots \oplus a_n$
- 

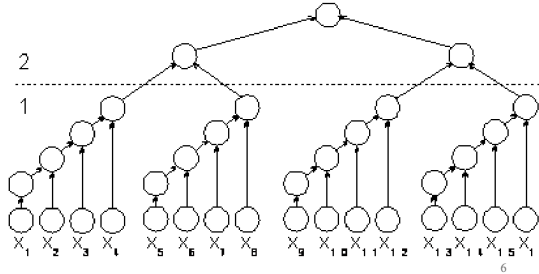
Алгоритм:

- Сначала входные данные делятся на группы по  $\log_2 n$  элементов
  - Всего  $\frac{n}{\log_2 n}$  групп
- Затем вычисления разбиваются на два этапа:
  1. В каждой группе операция применяется последовательно
  2. К результатам первого этапа применяется обычная схема сдваивания

# Модифицированная каскадная схема

В этом варианте каскадной схемы все проводимые вычисления подразделяется на два последовательно выполняемых этапа суммирования:

- на первом этапе вычислений все суммируемые значения подразделяются на  $(n/\log_2 n)$  групп, в каждой из которых содержится  $\log_2 n$  элементов; далее для каждой группы вычисляется сумма значений при помощи последовательного алгоритма суммирования; вычисления в каждой группе могут выполняться независимо друг от друга (т.е. параллельно – для этого необходимо наличие не менее процессоров  $(n/\log_2 n)$ );
- на втором этапе для полученных  $(n/\log_2 n)$  сумм отдельных групп применяется обычная каскадная схема.



В чём смысл модификации?

- В обычной каскадной схеме требуется  $\frac{n}{2} = O(n)$  вычислителей
- Здесь требуется  $\frac{n}{2 \log_2 n} = O(\frac{n}{\log n})$  вычислителей
- При том что и там, и там асимптотическая оценка алгоритма  $O(\log n)$

## 28. Оценка эффективности параллельных вычислений.

Две (противоречащих друг другу) цели при организации эффективных параллельных вычислений:

1. Максимизировать нагрузку на вычислители (минимизировать время простоя)
2. Минимизировать объем коммуникаций

Два теоретических закона для оценки эффективности:

- [Закон Амдала](#)
  - Позволяет оценить прирост производительности при фиксированном объеме задачи
- [Закон Густавсона-Барсиса](#)
  - Позволяет оценить прирост производительности при фиксированном времени работы

**Основные метрики** Ускорение --- отношение времени работы на одном вычислителе к времени работы на нескольких вычислителях

$$S_n := \frac{T_1}{T_n}$$

Эффективность --- отношение ускорения к числу вычислителей

$$E_n := \frac{S_n}{n}$$

**Производные метрики** Стоимость --- произведение количества вычислителей на их время работы

$$C_n := n * T_n$$

Избыточность --- отношение стоимости к времени работы на одном вычислителе

$$R_n := \frac{C_n}{T_1}$$

Накладные расходы --- разность стоимости и времени работы на одном вычислителе

$$O_n := C_n - T_1$$

**Гранулярность --- оценка объема коммуникаций** Гранулярность --- мера отношения объема вычислений, выполненных в параллельных вычислениях, к объему коммуникаций

CCR (Computation-to-Communication Ratio) --- метрика отношения времени вычислений ко времени коммуникации

Крупнозернистая

- Крупные, относительно независимые вычисления
- Редкие коммуникации
- Уровень задач для различных узлов вычислительного кластера
- Пример технологии: MPI

Среднезернистая

- Единица параллелизма --- вызываемые процедуры, состоящие из сотен-тысяч инструкций
- Периодические коммуникации
- Уровень задач для различных потоков управления на одной ОС
- Пример технологии: OpenMP

Мелкозернистая

- Вычисления уровня отдельных базовых блоков CFG
- Очень частый обмен данными
- Уровень задач для одного ядра CPU/GPU
- Пример технологий: конвейеризация, векторизация

## 29. Закон Амдаля.

Закон Амдала говорит, что если зафиксировать объем задачи и варьировать время, то прирост эффективности по мере роста числа вычислителей ограничен величиной, обратно пропорциональной доле вычислений, которые нельзя распараллелить

Пусть

- $s$  --- доля задачи, которую необходимо выполнять последовательно (нельзя распараллелить)
- $p$  --- доля задачи, которая идеально параллелится
- $T_1$  --- время выполнения задачи на одном вычислителе
- $T_n$  --- время выполнения задачи на  $n$  вычислителях

Тогда прирост производительности (ускорение) вычисляется следующим образом:

$$A := \frac{T_1}{T_n} = \frac{s + p}{s + \frac{p}{n}}$$

Если принять  $T_1$  за единицу, то

$$A = \frac{1}{s + \frac{p}{n}} = \frac{1}{s + \frac{1-s}{n}}$$

Например, если доля последовательных вычислений равна  $\frac{1}{5}$ , то можно посчитать количество вычислителей, которые понадобятся для того чтобы добиться прироста производительности в 4 раза:

$$4 = \frac{1}{\frac{1}{5} + \frac{1-\frac{1}{5}}{n}}$$

$$4 = \frac{1}{\frac{n+4}{5n}}$$

$$4 = \frac{5n}{n+4}$$

$$4n + 16 = 5n$$

$$n = 16$$

16 вычислителей потребуется, чтобы выполнить ту же задачу в 4 раза быстрее

---

Кроме того, прирост производительности при росте числа вычислителей можно ограничить сверху:

$$A = \frac{1}{s + \frac{1-s}{n}} \leq \frac{1}{s}$$

Например, если доля последовательных вычислений  $\frac{1}{5}$ , то получить прирост производительности больше чем в 5 раз не получится

### 30. Закон Густавсона-Барсиса

Закон Густавсона-Барсиса позволяет оценить прирост производительности при условии что время работы фиксировано, а объем задач варьируется

Пусть

- $s$  --- доля вычислений которые нельзя распараллелить
  - $p$  --- доля вычислений, которые идеально параллелятся
  - $L_1$  --- объем задачи на одном вычислителе
  - $L_n$  --- объем задачи на  $n$  вычислителях
- 

Если принять  $L_1$  за единицу, то можно вычислить увеличение объема задачи, который успевают выполнить вычислители, в зависимости от числа вычислителей

$$A := \frac{L_n}{L_1} = \frac{s + p * n}{s + p} = s + p * n$$

Например, чтобы получить прирост производительности в 4 раза при  $s = \frac{1}{4}$  понадобится 5 вычислителей

$$4 = \frac{1}{4} + \frac{3n}{4}$$

$$16 = 1 + 3n$$

$$n = 5$$

5 вычислителей потребуется, чтобы за то же время выполнить задачу, которая в 4 раза объемнее, чем исходная

## 31. Масштабируемость параллельных вычислений.

Закон Мура: каждые два года количество полупроводниковых элементов на единицу площади увеличивается в два раза, то есть растет экспоненциально

Закон Мура ``давал сбой`` дважды:

1. В ~2005 году
  1. До этого момента уменьшение размера транзистора приводило к тому, что его энергопотребление пропорционально падало
  2. Это позволяло наращивать частоту на одноядерном процессоре без повышения затрат на охлаждение
  3. Но потом технологии уперлись в барьер: около 4 ГГц, после которого энергопотребление растет полиномиально относительно роста частоты
  4. Наращивать тактовую частоту одноядерных процессоров стало невозможно без запредельного охлаждения
  5. Закончилась гонка частоты и началась гонка количества ядер
2. В ~2020 году
  1. Наращивание числа ядер перестало давать больших преимуществ
  2. Дальнейшее уменьшение техпроцесса уже было непозволительно дорогим

Вывод из закона Мура: сейчас программы с изменением процессоров сильно быстрее работать не начинают  
Решение --- писать параллельные программы

---

См. также [28. Оценка эффективности параллельных вычислений.](#)

## 32. Методика разработки параллельных алгоритмов.

Основные цели при разработке параллельных алгоритмов:

1. Максимально загрузить все доступные вычислители (минимизировать время простоя)
2. Минимизировать число коммуникаций (для синхронизации или обмена данными)

Задачи противоречат друг другу: чем сильнее распределяем задачи на вычислители, тем больше коммуникаций нужно

---

PCAM (Partitioning, Communication, Agglomeration, Mapping) --- подход к разработке параллельных алгоритмов, состоящий из 4 этапов

1. Partitioning
  1. Исходная задача разделяется на независимые подзадачи, которые можно выполнять параллельно
2. Communication
  1. Выделяются информационные зависимости между подзадачами
3. Agglomeration
  1. Группировка задач с целью минимизации накладных расходов на коммуникацию
  2. Объединяются в одну группу те задачи, между которыми есть активные коммуникации
4. Mapping
  1. Назначение групп задач конкретным вычислительным узлам

---

Также как отдельные этапы можно выделить:

- [Оценку эффективности](#)
- [Оценку масштабируемости](#)

### 33. Гравитационная задача n тел

Постановка задачи:

- Есть  $N$  материальных точек
  - У каждой точки своя масса, положение и скорость в текущий момент времени
  - Необходимо рассчитать положение тела в следующий момент времени, учитывая влияние остальных тел
- 

Алгоритм хорошо параллелится:

- В рамках одного шага вычисление положения одного тела не зависит от вычисления положения другого тела
- Исходная задача естественным образом разбивается на  $N$  независимых подзадач

При этом потребуются барьерная синхронизация и большое количество коммуникаций: когда очередной вычислитель закончил считать новое положение, он должен дожидаться завершения работы остальных и получить их результаты

---

Наивный подход:

- Для каждого тела рассчитываем влияние каждого другого тела на него
  - Асимптотика:  $O(N^2)$
- 

Алгоритм Барнса-Хата:

- Даёт приближенное значение
- Идея:
  - Влияние удаленных тел не так велико
  - Поэтому влияние скопления удаленных тел аппроксимируется влиянием одной материальной точки
    - \* Тела, которые находятся близко друг к другу, но далеко от текущей точки "стягиваются" в одну точку
    - \* Таких новых точек может получиться несколько
  - Масса точки --- сумма масс удаленных тел
  - Положение точки --- центр масс удаленных тел
- Асимптотика:  $O(N * \log N)$
- Также алгоритм требует динамической балансировки: тела могут быть распределены неравномерно, в скоплениях вычисления будут более трудоемкими, чем в регионах с низкой плотностью

### 34. Метрика, метрическое пространство, шар.

Метрика на  $X$  --- функция  $\rho : X \times X \rightarrow \mathbb{R}$ , удовлетворяющая следующему набору свойств:

1. Неотрицательность:
  1.  $\forall x, y \in X \quad \rho(x, y) \geq 0$
  2.  $\forall x, y \in X \quad \rho(x, y) = 0 \iff x = y$
2. Симметричность:
  1.  $\forall x, y \in X \quad \rho(x, y) = \rho(y, x)$
3. Неравенство треугольника:
  1.  $\forall x, y, z \in X \quad \rho(x, y) + \rho(y, z) \geq \rho(x, z)$

Пару  $(X, \rho)$  где  $X$  --- множество,  $\rho$  --- метрика на нём называют метрическим пространством

Примеры метрик:

- Дискретная:

$$\rho(x, y) = \begin{cases} 0, & x = y \\ 1, & x \neq y \end{cases}$$

- Метрика Минковского на  $\mathbb{R}^n$  для  $p \geq 1$ :

$$\rho(x, y) = \sqrt[p]{\sum_{k=1}^n |x_k - y_k|^p}$$

- Её частные случаи:

\* Манхэттенская метрика при  $p = 1$

\* Евклидова метрика при  $p = 2$

\* Метрика Чебышева при  $p = \infty$ :

$$\rho(x, y) = \max_{k=1..n} |x_k - y_k|$$

- sup-метрика для функций на отрезке  $[a, b]$

$$\rho(f, g) = \sup_{x \in [a, b]} |f(x) - g(x)|$$

(Открытым) шаром с центром в  $x \in X$  радиуса  $r \in \mathbb{R}$  называют

$$B_r(x) := \{y \in X : \rho(x, y) < r\}$$

Замкнутым шаром (или диском) с центром в  $x \in X$  радиуса  $r \in \mathbb{R}$  называют

$$\overline{B}_r(x) := \{y \in X : \rho(x, y) \leq r\}$$

## 35. Сходимость по метрике. Эквивалентные метрики. Открытое и замкнутое множество. Окрестность.

Пусть  $(X, \rho)$  --- метрическое пространство

Говорят что последовательность  $\{x_k\}_{k=1}^{\infty}$  сходится по метрике к  $x$  если

$$\lim_{k \rightarrow \infty} \rho(x_k, x) = 0$$

То есть

$$\forall \varepsilon > 0 \quad \exists N = N(\varepsilon) : \forall n \geq N \quad \rho(x, x_n) < \varepsilon$$

Говорят, что две метрики  $\rho_1, \rho_2$  на  $X$  эквивалентны, если любая последовательность сходится по метрике  $\rho_1$  тогда и только тогда когда эта последовательность сходится по метрике  $\rho_2$

Множество  $E$  называют открытым в  $X$ , если

$$\forall x \in E \cap X \quad \exists r > 0 : B_r(x) \subseteq E$$

где  $B_r(x)$  --- (открытый) шар радиуса  $r$  с центром в  $x$

Множество  $D \subseteq X$  называют замкнутым, если его дополнение является открытым

Окрестностью  $x \in X$  называют любое открытое в  $X$  множество, содержащее  $x$

**Альтернативные определения** Пусть  $X$  --- множество,  $\Omega \subseteq 2^X$  называют топологией, если

1.  $\emptyset, X \in \Omega$

2. Замкнутость относительно любого объединения:

$$1. \forall \mathcal{A} \subseteq \Omega \quad \bigcup_{A \in \mathcal{A}} A \in \Omega$$

3. Замкнутость относительно конечного пересечения:

$$1. \forall \{A_k\}_{k=1}^n \subseteq \Omega \quad \bigcap_{k=1}^n A_k \in \Omega$$



Пару  $(X, \Omega)$  называют топологическим пространством

---

Множество  $E \subseteq X$  называют открытым в  $(X, \Omega)$ , если оно является элементом топологии, то есть если  $E \in \Omega$

Множество  $D \subseteq X$  называют замкнутым в  $(X, \Omega)$ , если его дополнение является открытым, то есть если  $(X \setminus D) \in \Omega$

Окрестностью точки  $x \in X$  называют любое открытое множество  $E \in \Omega$ , содержащее  $x$

---

На метрическом пространстве  $(X, \rho)$  можно определить множество всех открытых шаров:

$$\mathcal{B} := \{ B_r^\rho(x) : x \in X, r > 0 \}$$

где  $B_r(x)$  --- открытый шар радиуса  $r$  с центром в точке  $x$ , построенный с помощью метрики  $\rho$

Всевозможные объединения открытых шаров будут являться топологией на  $X$ :

$$\Omega := \left\{ \bigcup_{U \in \mathcal{B}} U : B \subset \mathcal{B} \right\}$$

При этом говорят, что метрика  $\rho$  задаёт на  $X$  метрическую топологию  $\Omega$

---

Две метрики называют эквивалентными на  $X$ , если на  $X$  они задают одну и ту же топологию

### 36. Фундаментальная последовательность и ее свойства.

В метрическом пространстве  $(X, \rho)$  последовательность  $\{x_k\}_{k=1}^\infty$  называют фундаментальной последовательностью (или последовательностью Коши) если

$$\forall \varepsilon > 0 \quad \exists N = N(\varepsilon) : \forall n \geq N \quad \forall p \in \mathbb{N} \quad \rho(x_n, x_{n+p}) < \varepsilon$$

---

Теорема: Всякая сходящаяся (по метрике  $\rho$ ) последовательность является фундаментальной (по метрике  $\rho$ )

Доказательство: Пусть последовательность сходится к некоторому  $a \in X$ . Зафиксируем  $\varepsilon > 0$ , из определения сходимости следует что

$$\exists N = N(\varepsilon) : \forall n \geq N \quad \begin{cases} \rho(a, x_n) < \frac{\varepsilon}{2} \\ \rho(a, x_{n+p}) < \frac{\varepsilon}{2} \end{cases}$$

По неравенству треугольника:

$$\rho(x_n, x_{n+p}) \leq \rho(x_n, a) + \rho(a, x_{n+p}) < \frac{\varepsilon}{2} + \frac{\varepsilon}{2} = \varepsilon$$

---

Утверждение: Всякая фундаментальная последовательность ограничена. Доказательство: Пусть  $\varepsilon = 1$ , тогда по определению фундаментальности:

$$\exists N \in \mathbb{N} : \forall n \geq N \quad \rho(x_n, x_N) < 1$$

Зафиксируем произвольную точку  $a \in X$ . По неравенству треугольника:

$$\forall n \geq N \quad \rho(x_n, a) \leq \rho(x_n, x_N) + \rho(x_N, a)$$

Причём:

- $\rho(x_n, x_N) < 1$
- $\rho(x_N, a)$  --- конечная величина, не зависящая от  $n$

Поэтому все элементы последовательности, начиная с  $N$  окажутся в шаре с центром в  $a$  и радиусом  $r_1 := 1 + \rho(x_N, a)$

Все элементы последовательности до  $N$  окажутся в шаре с центром в  $a$  и радиусом  $r_2 := \max_{k=1..N} \rho(a, x_k)$

- Их конечное число, поэтому максимум достигается

Итого, все элементы последовательности окажутся в шаре с центром в  $a$  и радиусом  $\max(r_1, r_2)$

## 37. Полнота. Изометрия. Пополнение.

### Полнота

Метрическое пространство  $(X, \rho)$  называют полным, если в нём любая фундаментальная (по  $\rho$ ) последовательность сходится (по  $\rho$ )

Примеры полных метрических пространств:

- $\mathbb{R}^n$  с евклидовой метрикой
- Непрерывные на  $[a, b]$  функции с  $\sup_{[a,b]}$ -метрикой

### Изометрия

Отображение  $f : X \rightarrow Y$  где  $(X, \rho_1)$  и  $(Y, \rho_2)$  --- метрические пространства, называют изометрией, если

$$\forall x_1, x_2 \in X \quad \rho_1(x_1, x_2) = \rho_2(f(x_1), f(x_2))$$

Любая изометрия является инъекцией Два метрических пространства называют изометричными, если между ними существует изометрия, являющаяся биекцией

**Свойства** Между изометричными пространствами существует взаимно-однозначное соответствие, сохраняющее расстояния между точками Таким образом, все свойства множеств, выраженные в терминах метрик, сохраняются при изометрии

Примеры:

1. Если  $\{x_k\}_{k=1}^{\infty}$  сходится по метрике  $\rho_1$ , то  $\{f(x_k)\}_{k=1}^{\infty}$  сходится по метрике  $\rho_2$ 
  1. Аналогично для фундаментальности последовательности, равномерной сходимости функциональной последовательности, сходимости числовых и функциональных рядов
2. Если  $E \subseteq X$  открыто, то и  $f(E) \subseteq Y$  открыто
  1. Аналогично для замкнутости, ограниченности, компактности
3. Если  $(X, \rho_1)$  полно, то и  $(Y, \rho_2)$  полно

Кроме того, изометричность метрических пространств является отношением эквивалентности

### Пополнение

#### ТИП

Одно из возможных определений плотности

Говорят, что  $A \subseteq X$  плотно в метрическом пространстве  $(X, \rho)$ , если для любого открытого  $U \subset X$  верно что  $A \cap U \neq \emptyset$  Например,  $\mathbb{Q}$  плотно в  $\mathbb{R}$  --- какой бы вещественный открытый шар вы не взяли, в нём всегда найдется рациональное число Например,  $\mathbb{Z}$  не плотно в  $\mathbb{R}$  --- можно подобрать открытый шар, в котором не будет целых чисел, например  $(0, 1)$

Пополнением метрического пространства  $(X, \rho)$  называют пару  $\langle (\tilde{X}, \tilde{\rho}), i \rangle$  где

- $(\tilde{X}, \tilde{\rho})$  --- полное метрическое пространство
- $i : X \rightarrow \tilde{X}$  --- изометрия
  - И при этом  $i(X)$  плотно в  $\tilde{X}$

Мотивация:

- Пополнение "исправляет" неполное пространство, добавляя к нему недостающие пределы фундаментальных последовательностей
- При этом исходное пространство плотно вкладывается в получившееся
- При этом сохраняются все расстояния между точками за счёт того что  $i$  изометрия
  - И как следствие, все свойства, которые изометрия переносит

Теорема:

- У любого метрического пространства существует пополнение
- Кроме того, любые два пополнения метрического пространства изометричны
  - Другими словами, пополнение единственно с точностью до изометрии

### 38. Оператор. Непрерывность. Условие Липшица.

**Оператор** Рассматриваем два полных метрических пространства  $(X, \rho_1)$  и  $(Y, \rho_2)$

$A : X \rightarrow Y$  называют оператором если

$$\forall x \in X \quad \exists! y \in Y : Ax = y$$

То есть оператор --- произвольное отображение между доменами метрических пространств

**Функционал** Оператор называют функционалом если  $Y = \mathbb{C}$  или  $Y = \mathbb{R}$

- Подразумевается, что на  $\mathbb{C}$  или на  $\mathbb{R}$  задана стандартная метрика  $\rho(x, y) = |x - y|$
- Функционал --- оператор, ставящий в соответствие элементу домена скаляр

**Непрерывность** Говорят, что оператор  $A : X \rightarrow Y$ , заданный на метрических пространствах  $(X, \rho_1)$  и  $(Y, \rho_2)$  непрерывен в точке  $x$ , если

$$\lim_{t \rightarrow x} At = Ax$$

Говорят, что оператор  $A$  непрерывен, если он непрерывен в каждой точке, то есть если

$$\forall x \in X \quad \lim_{t \rightarrow x} At = Ax$$

Эквивалентное определение --- образ любой сходящейся последовательности сходится к образу предела этой последовательности:

$$x_n \xrightarrow{n \rightarrow \infty} x \implies Ax_n \xrightarrow{n \rightarrow \infty} Ax$$

Эквивалентное определение:

$$\forall x \in X \quad \forall \varepsilon > 0 \quad \exists \delta = \delta(\varepsilon, x) : \forall t \in X \quad \rho_1(x, t) < \delta \implies \rho_2(Ax, At) < \varepsilon$$

(Не путать с равномерной непрерывностью):

$$\forall \varepsilon > 0 \quad \exists \delta = \delta(\varepsilon) : \forall x, t \in X \quad \rho_1(x, t) < \delta \implies \rho_2(Ax, At) < \varepsilon$$

**Условие Липшица** Оператор  $A : X \rightarrow Y$  называют липшицевым, если

$$\exists L \in \mathbb{R} \forall x, t \in X \quad \rho_2(Ax, At) \leq L \rho_1(x, t)$$

При этом  $L \in \mathbb{R}$  называют константой Липшица

Условие Липшица --- более сильное условие, чем равномерная непрерывность:

$$\text{Липшицевость} \implies \text{Равномерная непрерывность} \implies \text{Непрерывность}$$

### 39. Неподвижная точка. Сжатие. Итерационный метод.

**Неподвижная точка** Пусть  $A : X \rightarrow X$  --- оператор  $x \in X$  называют неподвижной точкой оператора  $A$ , если  $Ax = x$

**Сжатие** Оператор  $A : X \rightarrow X$  называют оператором сжатия, если

1. Он является липшицевым оператором
2. Константа Липшица меньше единицы

То есть  $A$  является оператором сжатия, если

$$\exists L \in [0, 1) : \forall x, t \in X \quad \rho(Ax, At) \leq L\rho(x, t)$$

В частности, для операторов сжатия верно, что

$$\forall x, t \in X \quad \rho(Ax, At) < \rho(x, t)$$

При применении оператора расстояние между точками уменьшается

**Итерационный метод (последовательные приближения)** Пусть  $A : X \rightarrow X$  --- оператор Пусть  $\{x_k\}_{k=1}^{\infty}$  --- последовательность в  $X$  Говорят, что  $A$  --- итерационный метод (или метод последовательных приближений), если:

$$\forall k \in \mathbb{N} \quad x_{k+1} = Ax_k$$

В таком случае  $\{x_k\}_{k=1}^{\infty}$  называют итерационной последовательностью (или последовательностью приближений)

## 40. Принцип сжимающих отображений

**Теорема Банаха (Принцип сжимающих отображений)** Пусть

- $(X, \rho)$  --- полное метрическое пространство
- $A : X \rightarrow X$  --- оператор сжатия
- $F \subseteq X$  --- замкнутое множество
- $A(F) \subseteq F$

Тогда для оператора  $A$  существует единственная неподвижная точка в  $F$  (решение уравнения  $Ax = x$ )

**Нахождение неподвижной точки** Эта неподвижная точка находится как предел итерационной последовательности  $\{x_k\}_{k=0}^{\infty}$  (Каким-нибудь образом) выбирается  $x_0 \in F$ , после чего, посредством последовательного применения, получаются следующие точки:  $x_{k+1} = Ax_k$

В силу того, что оператор  $A$  уменьшает расстояние между  $x_k$  и  $x_{k+1}$  при очередном применении, рано или поздно, он "сожмёт" всю последовательность в неподвижную точку (в одну и ту же точку, вне зависимости от выбора  $x_0$ )

Универсального метода выбора  $x_0$  нет Если  $F$  --- замкнутый шар, то можно взять его центр в качестве  $x_0$

## 42. О приближенном решении нелинейных уравнений.

Допустим, дано нелинейное уравнение  $f(x) = 0$  В силу того, что аналитически найти решение получается только для узкого класса задач, в остальных случаях используют численные методы

---

Прямое применение теоремы Банаха:

1. Нужно выбрать множество, на котором решение гарантировано существует и единственно
  2. Из уравнения  $f(x) = 0$  нужно выразить  $x$ , получив уравнение вида  $x = \varphi(x)$
  3. Если  $\varphi$  является сжимающим оператором на выбранном отрезке, то можно по теореме Банаха найти его неподвижную точку  $x^*$
  4. Эта неподвижная точка и будет решением исходного уравнения  $f(x) = 0$
-

Применение метода Ньютона:

1. В качестве итерационной функции берётся  $\varphi(x) = x - \frac{f(x)}{f'(x)}$ 
  1. То есть  $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$
2. В неподвижной точке  $x^* = \varphi(x^*) = x^* - \frac{f(x^*)}{f'(x^*)}$ , следовательно  $f(x^*) = 0$  и  $x^*$  будет искомым корнем
  - Метод может применяться до тех пор, пока производная в точке приближения не обращается в ноль
    - Если  $f'(x_n) = 0$ , то найти  $x_{n+1}$  уже не удастся
    - Можно попробовать выбрать другое начальное приближение --- оно может дать более точный результат
  - Для гарантированной сходимости  $\{x_n\}_{n=1}^{\infty}$  к корню  $f(x) = 0$  нужны дополнительные условия (например условия Фурье)

## 43. Метод половинного деления

Один из методов нахождения приближенных корней уравнения  $f(x) = 0$  на отрезке  $[a, b]$

Контекст:

- $f$  непрерывна на  $[a, b]$
  - На отрезке  $[a, b]$  корень существует и единственен
    - То есть  $f(a)$  и  $f(b)$  разных знаков и  $\exists c \in (a, b) : f(c) = 0$
- 

Преимущества:

- Простой
- Сходится всегда

Недостаток:

- Медленный --- линейно (по количеству знаков после запятой) сходится к правильному ответу
  - Для сравнения: метод Ньютона сходится квадратично: количество правильных знаков после запятой удваивается при каждом последующем приближении
- 

Сущность:

1. На очередном шаге рассматриваем середину отрезка:  $c := \frac{a+b}{2}$
2. Если  $f(c) = 0$ , то мы нашли корень
3. Иначе если  $f(a)$  и  $f(c)$  разных знаков, то корень лежит в левой половине отрезка: в качестве следующего отрезка выберем  $[a, c]$
4. Иначе если  $f(c)$  и  $f(b)$  разных знаков, то корень лежит в правой половине отрезка: в качестве следующего отрезка выберем  $[c, b]$